

Liste avanzate

Liste avanzate

List comprehension e tecniche avanzate per le liste in Python.

Perché è utile usare le liste avanzate

Sai già usare le liste di base. Ora impari le tecniche che rendono il codice Python **più corto, più leggibile e più efficiente** — strumenti che i programmatori Python usano ogni giorno.

List comprehension: creare liste in una riga

La **list comprehension** è un modo compatto per costruire una lista a partire da un'altra sequenza, tutto in una sola riga.

Il problema che risolve

```
# Metodo tradizionale: 4 righe per creare una lista di quadrati
quadrati = []
for x in range(1, 6):
    quadrati.append(x ** 2)
print(quadrati)  # [1, 4, 9, 16, 25]

# Con list comprehension: 1 sola riga, stesso risultato
quadrati = [x ** 2 for x in range(1, 6)]
print(quadrati)  # [1, 4, 9, 16, 25]
```

Leggi la list comprehension come: "x al quadrato, per ogni x in range(1, 6)".

Con un filtro (**if**)

Puoi aggiungere una condizione per includere solo certi elementi:

```
# Solo i numeri pari tra 0 e 9
pari = [x for x in range(10) if x % 2 == 0]
print(pari)    # [0, 2, 4, 6, 8]

# Solo le parole di più di 4 lettere
parole = ["mela", "banana", "uva", "ciliegia", "kiwi"]
lunghe = [p for p in parole if len(p) > 4]
print(lunghe)  # ['banana', 'ciliegia']
```

Con **if-else** (trasformazione condizionale)

Puoi trasformare ogni elemento in modo diverso a seconda di una condizione:

```
# Classifica ogni numero come "pari" o "dispari"
numeri = range(6)
classificati = ["pari" if x % 2 == 0 else "dispari" for x in numeri]
print(classificati)  # ['pari', 'dispari', 'pari', 'dispari', 'pari', 'dispari']
```

List comprehension annidate (per le matrici)

Puoi usare una list comprehension dentro un'altra per lavorare con strutture a griglia (righe e colonne):

```
# Creare una matrice 3x3 (lista di liste)
matrice = [[j for j in range(3)] for i in range(3)]
print(matrice)  # [[0, 1, 2], [0, 1, 2], [0, 1, 2]]

# "Appiattare" una matrice in una lista semplice
matrice = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
piatta = [elem for riga in matrice for elem in riga]
print(piatta)  # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Ordinare con `sorted()` e una chiave personalizzata

Per default `sorted()` ordina in ordine alfabetico o numerico. Con il parametro `key` puoi scegliere tu **in base a cosa** ordinare.

```
studenti = [("Alice", 16), ("Carlo", 15), ("Bob", 17)]

# Ordina per età (il secondo elemento della coppia)
per_eta = sorted(studenti, key=lambda s: s[1])
print(per_eta)  # [('Carlo', 15), ('Alice', 16), ('Bob', 17)]

# Ordina per nome (il primo elemento della coppia)
per_nome = sorted(studenti, key=lambda s: s[0])
print(per_nome)  # [('Alice', 16), ('Bob', 17), ('Carlo', 15)]

# Ordine dal più grande al più piccolo
decescente = sorted(studenti, key=lambda s: s[1], reverse=True)
print(decescente)  # [('Bob', 17), ('Alice', 16), ('Carlo', 15)]
```

map() e filter() : applicare funzioni a liste

```
numeri = [1, 2, 3, 4, 5]

# map(): applica una funzione a ogni elemento e restituisce i risultati
doppi = list(map(lambda x: x * 2, numeri))
print(doppi)  # [2, 4, 6, 8, 10]

# filter(): tieni solo gli elementi che soddisfano la condizione
grandi = list(filter(lambda x: x > 3, numeri))
print(grandi)  # [4, 5]
```

:::note Spesso le list comprehension sono più leggibili di `map` e `filter`. Usa quella forma che trovi più chiara. :::

zip() : scorrere più liste insieme

`zip()` combina due o più liste in coppie (o triplete, ecc.), permettendoti di scorrerle tutte insieme.

```
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]
citta = ["Roma", "Milano", "Napoli"]

# Scorrere due liste in parallelo
for nome, voto in zip(nomi, voti):
    print(f"{nome}: {voto}")
# → Alice: 8
# → Bob: 7
# → Carlo: 9

# Creare una lista di coppie
coppie = list(zip(nomi, voti, citta))
print(coppie)
# → [('Alice', 8, 'Roma'), ('Bob', 7, 'Milano'), ('Carlo', 9, 'Napoli')]
```

Operazioni comuni sulle liste

```
numeri = [3, 1, 4, 1, 5, 9, 2, 6]

print(sum(numeri))      # 31 – somma di tutti i valori
print(max(numeri))      # 9 – il valore più grande
print(min(numeri))      # 1 – il valore più piccolo
print(len(numeri))      # 8 – quanti elementi ci sono

# Calcolare la media
media = sum(numeri) / len(numeri)
print(f"Media: {media:.2f}") # Media: 3.88

# Rimuovere i duplicati mantenendo l'ordine originale
unici = list(dict.fromkeys(numeri))
print(unici) # [3, 1, 4, 5, 9, 2, 6]
```